Rebuild Creative Portal from Scratch

Complete prompt to regenerate the Univest Performance Marketing Portal. Feed this to Claude Code or any capable AI coding assistant.

OBJECTIVE

Build a **7-tab performance marketing portal** for Univest (a fintech app) that combines Meta Ads management, Google Ads management, competitor intelligence, creative analytics, trend scanning, AI-powered optimization, and a self-learning feedback engine — all in one unified web application.

ENVIRONMENT & API KEYS

Create a `.env` file in the project root with the following keys:

```
# Meta (Facebook) Ads API
META_ACCESS_TOKEN=<your-meta-access-token>
META_APP_SECRET=<your-meta-app-secret>
META_APP_SECRET_PROOF=<your-app-secret-proof>
META_AD_ACCOUNT_ID=act_725019929189148

# Google Apps Script (for Google Sheets read/write)
GOOGLE_APPS_SCRIPT_URL=https://script.google.com/macros/s/<your-script-id>/exec

# OpenAI (for AI analysis, classification, concept generation)
OPENAI_API_KEY=sk-proj-<your-openai-key>

# Metabase (internal analytics database)
METABASE_SESSION_TOKEN=<your-metabase-session-token>
```

External Services

Service	URL	Purpose
Meta Graph API	`https://graph.facebook.com/v21.0/`	Ad insights, ad management, media upload, ad library
Metabase	`https://analytics.univest.in/api/dataset`	SQL queries against Prod PostgreSQL (database_id=2) for funnel metrics
OpenAI	`https://api.openai.com/v1/chat/completions`	GPT-4 for ad classification, analysis, proposals, trend synthesis
Google Apps Script	(URL in .env)	Google Sheets read/write for ad specs

TECH STACK

- **Frontend:** Vanilla JavaScript (no React/Vue/Angular), HTML, CSS (dark theme)
- **Backend:** Node.js + Express.js
- **Database:** SQLite via `better-sqlite3` (one DB per module)
- **Scheduling:** `node-cron` for automated pipelines
- **Styling:** CSS Grid/Flexbox, dark theme with `#6c5ce7` purple and `#00d4aa` teal accents

Dependencies (package.json)

```
{
  "name": "performance-marketing-auto",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node uploader/server.js",
    "start:intel": "node intelligence/server.js",
    "start:all": "concurrently \"node uploader/server.js\" \"node intelligence/server.js\""
  },
  "engines": { "node": ">=18.0.0" },
  "dependencies": {
    "axios": "^1.7.0",
    "better-sqlite3": "^12.8.0",
    "cheerio": "^1.0.0",
    "concurrently": "^9.2.1",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "form-data": "^4.0.2",
    "google-ads-api": "^23.0.0",
    "node-cron": "^3.0.3",
    "openai": "^4.78.0",
    "rss-parser": "^3.13.0"
  }
}
```

ARCHITECTURE OVERVIEW

```
index.html (Tab Shell - 7 iframes)
  +-- Tab 1: Meta Portal         -> creative.html + app.js
  +-- Tab 2: Meta Ad Upload      -> uploader/upload.html + uploader/server.js
  +-- Tab 3: Inventory Scanner   -> inventory-scanner/public/index.html
  +-- Tab 4: Competitor Intel    -> inventory-scanner/public/ci.html
  +-- Tab 5: Google Portal       -> google-creative.html + google-creative/server.js
  +-- Tab 6: Trend Scanner       -> trend-scanner.html + trend-scanner/routes.js
  +-- Tab 7: Self-Learning       -> feedback.html + feedback-engine/server.js

Backend Servers:
  +-- uploader/server.js          (port 3000) - Main server
```

```
  +-- creative-intelligence/      - CI pipeline (mounted on main server)
  +-- feedback-engine/server.js   (port 3002) - Self-learning engine
  +-- google-creative/server.js   (port 3003) - Google Ads intelligence
  +-- inventory-scanner/server.js (port 3004) - Inventory discovery
  +-- trend-scanner/              - Trend pipeline (mounted on main server)

Databases (SQLite):
  +-- creative-intelligence/ci.db
  +-- feedback-engine/db/feedback-engine.db
  +-- google-creative/db/google-creative.db
  +-- inventory-scanner/database/scanner.db
```

TAB 1: META CREATIVE PORTAL

Purpose

Full-featured dashboard for monitoring Meta (Facebook + Instagram) ad creative performance.

Frontend: creative.html + app.js

Sidebar Navigation (11 views):

1. **Dashboard** - KPI grid (14 metrics) + summary charts
2. **All Creatives** - Full table of every creative with 20+ columns
3. **New This Week** - Creatives launched in last 7 days
4. **Top Performers** - Sorted by D6 ROAS descending
5. **Underperformers** - D6 ROAS < 15%, flagged for pause
6. **Scorecard** - Performance matrix (Exceptional / Performed / Try / Failed / Drop)
7. **Alerts** - Red/yellow/green flags for anomalies
8. **Campaign Tree** - Hierarchical: Campaign -> Adset -> Ad
9. **AI Intelligence** - GPT-4 analysis of portfolio
10. **ROAS Simulator** - Predict outcomes of budget changes
11. **Recommendations** - AI-generated optimization suggestions

14 KPI Metrics: Total Creatives, Live Count, Total Spend, Total Installs, CPI, CTR, Total Signups, Signup Cost, D6 ROAS, Overall ROAS, D0 Trial Cost, D0 CAC, D6 CAC, P0P1 Cost

Data Flow

Step 1 - Fetch Meta Insights:

```
POST /api/meta/ad-insights-daily
Body: { date_from, date_to }
-> Calls Meta Graph API: GET /v21.0/act_{AD_ACCOUNT_ID}/insights
   Fields: campaign_name, adset_name, ad_name, ad_id, spend, impressions, clicks,
           actions (mobile_app_install), video_p25/p50/p75/p100_watched_actions,
           video_play_actions, video_3s_watched
```

```
   Level: ad, Time increment: 1 (daily)
-> Returns: Array of daily ad-level metrics
```

Step 2 - Fetch Metabase Funnel:

```
POST /api/metabase/ad-funnel
Body: { date_from, date_to }
-> Calls Metabase: POST https://analytics.univest.in/api/dataset
   Headers: { "X-Metabase-Session": METABASE_SESSION_TOKEN }
   Body: {
     database: 2,
     type: "native",
     native: {
       query: "SELECT date, campaign, adset, tracker_name,
                     signups, d0_trials, d0_conversions, d0_revenue,
                     d6_conversions, d6_revenue, d6_overall_revenue,
                     overall_conversions, overall_revenue
               FROM ad_funnel_daily
               WHERE date BETWEEN '{{date_from}}' AND '{{date_to}}'
               AND platform = 'android'"
     }
   }
-> Returns: Rows of daily funnel metrics per campaign/adset/tracker
```

Step 3 - Fetch Ad Status:

```
GET /api/meta/ads-status
-> Calls Meta Graph API: GET /v21.0/act_{AD_ACCOUNT_ID}/ads
   Fields: id, name, effective_status
   Filtering: [{field:'effective_status', operator:'IN', value:['ACTIVE','PAUSED']}]
-> Returns: Map of ad_id -> status
```

Step 4 - Merge & Aggregate (in app.js):

- Group daily rows by creative (using ad_name as key)
- Sum: spend (×1.18 for GST), installs, clicks, impressions, video metrics
- Match Metabase rows to Meta rows using campaign+adset+tracker_name lookup
- Sum: signups, d0_trials, d0_conversions, d6_conversions, d6_revenue, d6_overall_revenue, overall_revenue
- Compute derived metrics: CPI, CTR, hook%, hold%, signup%, D6 ROAS, overall ROAS

AI Features (ai-cache.js + ai-context.js)

AI Context Builder packages all creative data into structured context:

- Portfolio benchmarks (median CPI, ROAS, signup rate across all creatives with >₹5K spend)
- Top 20 and Bottom 20 creatives by D6 ROAS (with >₹10K spend)
- Video-specific benchmarks (hook%, hold%, completion%)
- Static-specific benchmarks (CTR, CPI)
- LTV data (revenue per signup, D6->overall multiplier)
- Univest context (company=Univest, product=fintech investing app, target_audience=Indian retail investors)

```
POST /api/ai/analyze
Body: { system_prompt, user_prompt }
-> Calls OpenAI GPT-4: model "gpt-4", temperature 0.3, response_format: json_object
-> Returns: Structured JSON analysis
```

Caching: 4-hour TTL in localStorage + in-memory, force refresh option, minimum data requirements (3 creatives with spend, 2 with D6 data).

TAB 2: META AD UPLOADER

Purpose

Bulk upload video/static ad creatives to Meta from Google Sheets specs.

Frontend: uploader/upload.html

- Google Sheets sync button -> fetches ad specs
- Preview table showing all rows before upload
- Per-row execute button + bulk upload
- Status tracking (success/error per row)
- Write-back to Google Sheets with ad_id, adset_id, status

Backend: uploader/server.js

```
POST /api/upload/sheet-to-meta    -> Parse Google Sheet -> Upload ads
POST /api/upload/preview          -> Preview rows before upload
POST /api/upload/:index/execute   -> Upload single row
GET  /api/upload/status           -> Check upload progress
POST /api/upload/sync-from-sheets -> Fetch fresh sheet data
```

Upload Pipeline:

1. Read ad specs from Google Sheets (via Apps Script URL)
2. For each row: Upload media -> Create AdCreative -> Create/reuse AdSet -> Create Ad -> Write results back

```
POST /v21.0/{AD_ACCOUNT_ID}/advideos       -> Upload video
POST /v21.0/{AD_ACCOUNT_ID}/adimages       -> Upload image
POST /v21.0/act_{AD_ACCOUNT_ID}/adcreatives -> Create creative
POST /v21.0/act_{AD_ACCOUNT_ID}/adsets      -> Create adset
POST /v21.0/act_{AD_ACCOUNT_ID}/ads         -> Create ad
```

TAB 3: INVENTORY SCANNER

Purpose

Discover and track ad inventory placements, pricing, and format recommendations.

```
GET  /api/inventories              -> List tracked ad inventories
GET  /api/discovery                -> Discovery status & logs
POST /api/discovery/run            -> Trigger discovery scan
GET  /api/competitors              -> Competitor profiles
GET  /api/competitors/whitespace   -> Inventory gaps
GET  /api/budget/:inventoryId      -> Budget recommendations
GET  /api/pricing/:inventoryId     -> CPM/CPC benchmarks
GET  /api/formats/:inventoryId     -> Best-performing formats
```

Multi-Agent System: discoveryAgent, existingInventoryAgent, budgetAgent, pricingAgent, adFormatAgent, onboardingAgent, schedulerAgent

---

TAB 4: COMPETITOR INTELLIGENCE

Purpose

Monitor competitor ad activity via Meta Ad Library + classify ads + detect trends.

competitor-meta-fetcher.js - Fetch competitor ads:

```
GET https://graph.facebook.com/v19.0/ads_archive
  search_terms={competitor_name}
  ad_reached_countries=IN
  fields=id,ad_creative_body,ad_creative_link_caption,
         ad_delivery_start_time,ad_delivery_stop_time,
         impressions,spend,publisher_platforms
  limit=50 (paginated, max 10 pages / 500 ads per competitor)
```

Tracked Competitors: Zerodha, Groww, Angel One, 5Paisa, Upstox, INDmoney, Dhan, StockGro, Coin by Zerodha + Univest

competitor-classifier.js - Classify each ad:

- **Themes:** FOMO, Social Proof, Education, Offer/Discount, Feature, Comparison, Trust/SEBI, Free Trial, Testimonial, Celebrity
- **Formats:** Video, Static, Carousel, Story
- **Hook Styles:** Question, Stat-led, Benefit-led, Fear-led, Story-led, Offer-led
- **CTA Types:** App Install, Sign Up, Free Trial, Learn More, Call Now, Download
- **Audience Signals:** Beginner, Experienced, F&O Trader, Long-term, HNI

Strategy: Fast keyword match first, OpenAI GPT fallback for ambiguous ads.

competitor-trends.js - Compute signals: Rising/falling themes (14-day comparison), dominant format per competitor, evergreen ads (30+ days), new ads (7d), dormant competitors, platform diversification.

TAB 5: GOOGLE ADS PORTAL

Purpose

Dashboard for Google Ads creative performance (UAC, Search, PMax, Video, Display).

Identical layout to Meta portal. Database: SQLite `google-creative.db` (tables: gc_creatives, gc_creative_scores, gc_adset_performance).

Multi-Agent System: gcDataFetcher, gcClassifier, gcScoring, gcSignals, gcSimulator, gcForecast, gcRecommendations

```
GET  /creatives               -> List creatives with GCPS score
GET  /adsets                  -> Adset performance aggregation
POST /fetch/trigger           -> Manual data refresh
GET  /signals/:creativeId     -> Creative signals breakdown
GET  /recommendations         -> Batch recommendations
POST /simulator/run           -> Simulate budget changes
GET  /forecast                -> Revenue forecast
```

TAB 6: TREND SCANNER

Purpose

Real-time trend aggregation from 6 sources -> synthesize themes -> generate creative concepts.

9 Agents:

1. redditAgent - Stock market/investing subreddits
2. youtubeAgent - India finance YouTube channels
3. newsAgent - Financial news headlines
4. twitterAgent - India Twitter trending
5. googleTrendsAgent - Google Trends India
6. indiaFinanceAgent - India fintech content
7. synthAgent - Synthesize trends into themes (OpenAI)
8. scriptAgent - Generate creative concepts (static + video)
9. schedulerAgent - Scheduled scans

```
GET  /api/trends/status                  -> Scan progress
GET  /api/trends/data                    -> Synthesized trends + concepts
POST /api/trends/scan                    -> Trigger full scan
POST /api/trends/concepts/:trendId/:format -> Generate concept
GET  /api/trends/raw                     -> Raw source counts
```

```
Pipeline: Raw Trends (6 sources) -> Synthesized Themes -> Creative Concepts
```

TAB 7: SELF-LEARNING FEEDBACK ENGINE

Purpose

Autonomous AI system: monitor creatives, generate predictions, track accuracy, propose optimizations with human-in-the-loop approval.

8 Agents: feWatcher, feKnowledgeCrawler, feAccuracyAuditor, feSelfTester, feProposalGenerator, feApprovalEngine, feMemory, feAnomalyDetector

```
GET  /watcher/summary              -> Live predictions summary
POST /watcher/refresh              -> Force refresh
GET  /knowledge/items              -> Knowledge base search
POST /knowledge/crawl              -> Index new sources
GET  /audit/report                 -> Accuracy audit results
POST /audit/run                    -> Run new audit
GET  /hypotheses                   -> List test hypotheses
POST /hypotheses/generate          -> Generate new hypotheses
GET  /proposals                    -> List AI proposals
POST /proposals/generate           -> Generate new proposals
POST /proposals/:id/approve        -> Approve proposal
POST /proposals/:id/reject         -> Reject with reason
POST /proposals/approve-batch      -> Bulk approve
GET  /memory/episodic              -> Event logs
GET  /memory/semantic              -> Learned patterns
GET  /memory/procedural            -> Workflow docs
GET  /anomalies                    -> Detected anomalies
POST /anomalies/:id/resolve        -> Mark resolved
```

CREATIVE INTELLIGENCE PIPELINE

Continuous monitoring + action generation. Database: SQLite `ci.db` (tables: simulations, snapshots, trends, actions, analyses, pipeline_runs).

Pipeline Stages:

1. **Collect** - Daily snapshots from Meta + Metabase
2. **Analyze** - Compute trends, detect anomalies, run predictions
3. **Action** - Generate pause/scale/watch/kill recommendations
4. **Forecast** - D6/D30/D60/D120/D365 ROAS predictions

```
POST /pipeline/run             -> Trigger full pipeline (async)
GET  /pipeline/status          -> Pipeline status + scheduler
GET  /dashboard                -> Latest snapshots, actions, trends, predictions
GET  /snapshots                -> Daily creative snapshots
GET  /trends/:adId             -> Metric trends (7d/30d/all-time)
GET  /actions                  -> Active pause/scale/watch/kill actions
POST /actions/:id/execute      -> Execute action via Meta API
GET  /predictions              -> ROAS predictions at multiple horizons
```

OPTIMIZER MODULE

Scan -> Plan -> Execute workflow for campaign optimization.

- **PAUSE:** D6 ROAS < threshold AND spend > minimum AND days live > maturity
- **SCALE:** D6 ROAS > threshold AND sufficient volume AND under budget cap
- **ACTIVATE:** Ad currently paused but meets performance criteria
- **NO_ACTION:** Hold for observation (insufficient data)

KEY METRICS & CALCULATIONS

Univest Funnel

```
Impression -> Click -> Install -> Signup -> D0 Trial -> D0 Payment -> D6 Payment -> Overall Revenue
```

- **D0 Trial:** Trial started within 24h of install
- **D0:** Payment on day 0
- **D6:** Payment within 6 days (PRIMARY conversion metric)
- **Overall:** Lifetime revenue (all payments ever)

KPI Formulas

Metric	Formula
CPI	Spend / Installs
CTR	(Clicks / Impressions) x 100
Signup Cost	Spend / Signups
D0 Trial Cost	Spend / D0 Trials
D6 CAC	Spend / D6 Conversions
D6 ROAS	(D6 Revenue / Spend) x 100
Overall ROAS	(Overall Revenue / Spend) x 100
Hook Rate	(3-sec Video Views / Impressions) x 100
Hold Rate	(Thruplay / 3-sec Views) x 100
Signup %	(Signups / Installs) x 100

Performance Rating Thresholds

Rating	D6 ROAS
Exceptional	> 40%

Performed	28-40%
Try	15-28%
Failed	< 15%
Drop	Declining trajectory (slope < -10% over 7d)

Spend Adjustment: All Meta spend is multiplied by **1.18** (18% GST) to reflect true cost.

METABASE INTEGRATION

```
const metabaseUrl = 'https://analytics.univest.in/api/dataset';
const headers = {
  'Content-Type': 'application/json',
  'X-Metabase-Session': process.env.METABASE_SESSION_TOKEN
};

const body = {
  database: 2,  // Prod PostgreSQL
  type: "native",
  native: {
    query: "YOUR SQL HERE",
    "template-tags": {}
  }
};

// Response parsing:
const result = await axios.post(metabaseUrl, body, { headers });
const rows = result.data.data.rows;
const cols = result.data.data.cols.map(c => c.name);
```

Key Tables: ad_funnel_daily (signups, trials, conversions, revenue per campaign/adset/tracker), cohort data (P0/P1 subscribers), time-series (daily conversions/revenue).

META API INTEGRATION

Base URL: `https://graph.facebook.com/v21.0/`

Auth: `access_token={META_ACCESS_TOKEN}` as query parameter.

```
# Daily ad insights
GET /act_{AD_ACCOUNT_ID}/insights
  ?level=ad&time_increment=1
  &fields=campaign_name,adset_name,ad_name,ad_id,spend,impressions,clicks,
          actions,video_p25_watched_actions,video_p50_watched_actions,
          video_p75_watched_actions,video_p100_watched_actions,
          video_play_actions,video_3s_watched
  &time_range={"since":"YYYY-MM-DD","until":"YYYY-MM-DD"}
  &filtering=[{"field":"campaign.name","operator":"CONTAIN","value":"android"}]
```

```
# Ad status
GET /act_{AD_ACCOUNT_ID}/ads
  ?fields=id,name,effective_status&limit=500

# Ad Library (competitor research)
GET /ads_archive?search_terms={name}&ad_reached_countries=IN&limit=50

# Media upload
POST /{AD_ACCOUNT_ID}/advideos    (multipart/form-data)
POST /{AD_ACCOUNT_ID}/adimages    (multipart/form-data)

# Create objects
POST /act_{AD_ACCOUNT_ID}/adcreatives
POST /act_{AD_ACCOUNT_ID}/adsets
POST /act_{AD_ACCOUNT_ID}/ads

# Update ad status
POST /{AD_ID} { status: "PAUSED" | "ACTIVE" }
```

STYLING GUIDELINES

```
--bg-primary: #0a0a1a;
--bg-secondary: #1a1a2e;
--bg-card: #16213e;
--text-primary: #e0e0e0;
--text-secondary: #a0a0b0;
--accent-purple: #6c5ce7;
--accent-teal: #00d4aa;
--accent-red: #ff4757;
--accent-yellow: #ffa502;
--accent-green: #2ed573;
--border: rgba(255,255,255,0.08);
```

Layout: Sidebar navigation (collapsible) + main content area, responsive CSS Grid. Sortable tables with sticky headers. Rounded card corners with hover shadow.

FILE STRUCTURE

```
creative-portal/
+-- .env
+-- package.json
+-- index.html                        # Tab shell (7 iframes)
+-- creative.html                     # Meta portal frontend
+-- google-creative.html              # Google Ads portal frontend
+-- feedback.html                     # Self-learning frontend
+-- trend-scanner.html                # Trend scanner frontend
+-- styles.css                        # Shared styles
+-- app.js                            # Meta portal logic
```

```
+-- ai-cache.js                        # AI result caching (4hr TTL)
+-- ai-context.js                      # AI context builder
+-- optimizer.js                       # Scan/Plan/Execute optimizer
+-- competitor-meta-fetcher.js         # Meta Ad Library fetcher
+-- competitor-classifier.js           # Ad classification
+-- competitor-trends.js               # Trend signals
+-- competitor-radar.js                # Competitor radar
+-- competitor-scheduler.js            # Competitor scan cron
+-- competitor-routes.js               # Competitor API routes
+-- uploader/
|   +-- server.js                      # Main Express server (port 3000)
|   +-- upload.html                    # Upload frontend
|   +-- meta-write.js                  # Meta API write helpers
+-- creative-intelligence/
|   +-- db.js, engine.js, predictor.js, routes.js
+-- feedback-engine/
|   +-- server.js                      # Express server (port 3002)
|   +-- agents/ (8 agents), db/, public/
+-- google-creative/
|   +-- server.js                      # Express server (port 3003)
|   +-- agents/ (7 agents), db/, public/
+-- inventory-scanner/
|   +-- server.js                      # Express server (port 3004)
|   +-- routes.js, agents.js, database/, public/
+-- trend-scanner/
    +-- routes.js, cache.js, agents/ (9 agents)
```

BUILD ORDER

1. `.env` + `package.json` - Environment setup
2. `uploader/server.js` - Main Express server with Meta API proxy + Metabase proxy
3. `index.html` + `styles.css` - Portal shell with 7 tabs
4. `creative.html` + `app.js` + `ai-cache.js` + `ai-context.js` - Meta portal
5. `uploader/upload.html` + `meta-write.js` - Ad uploader
6. `creative-intelligence/` - CI pipeline
7. `competitor-*.js` + `competitor-routes.js` - Competitor intel
8. `inventory-scanner/` - Inventory scanner
9. `google-creative/` - Google Ads portal
10. `trend-scanner/` - Trend scanner
11. `feedback-engine/` - Self-learning engine
12. `optimizer.js` - Campaign optimizer

BUSINESS CONTEXT

- **Company:** Univest - Indian fintech app for retail investors

- **Product:** Stock investing, mutual funds, IPOs, market insights
- **Target Audience:** Indian retail investors (age 18-45, Android-first)
- **Campaign Filter:** Only Android campaigns (filter by name containing "android")
- **Currency:** INR, all spend x1.18 for GST
- **Primary KPI:** D6 ROAS (revenue within 6 days of install / spend)
- **Mature KPI:** Overall ROAS (lifetime revenue / spend)
- **Decision Threshold:** Minimum 5K-10K spend before judging a creative